

Novel Pipeline Vulnerability Discovered; Rust Found Vulnerable

Novel Pipeline Vulnerability Discovered; Rust Found Vulnerable - Noam Dotan, Legit Security.

- Published: 2022-12-01
- Original: <<https://www.legitsecurity.com/blog/artifact-poisoning-vulnerability-discovered-in-rust>>
- Preserved from: <https://www.legitsecurity.com/blog/artifact-poisoning-vulnerability-discovered-in-rust> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Novel Pipeline Vulnerability Discovered; Rust Found Vulnerable

[Book a Demo](#)

The Legit Security Research Team discovered a new class of software supply chain vulnerabilities that leverages artifact poisoning and attacks the underlying software development pipelines for projects using GitHub Actions. In this fourth blog covering vulnerable GitHub Actions, we will explore this new technique of artifact poisoning and describe who could be vulnerable, including how we found this vulnerability in the Rust programming** language and assisted in its remediation.**

We have discovered that when transferring artifacts between different workflows, there is a major risk for *artifact poisoning – *a technique in which attackers replace the content of a legitimate artifact with a modified malicious one and thereby initiate a supply chain attack.

This is similar to the CodeCov supply chain breach, where attackers put a backdoor into one of Codecov’s artifacts which were later used by users to update their software and thereby got hacked.

This blog will discuss the GitHub Actions artifact storage, a limitation in the artifact storage, how GitHub Actions users overcame this limitation, and why this workaround opens the door for **Artifact Poisoning**.

If you haven't had the chance to read our previous posts in the series, you can find them [here](#):

-

[Part 1: Privilege Escalation Inside Your CI/CD Pipeline](#)

-

[Part 2: Actions That Open the Door to CI/CD Pipeline Attacks](#)

-

[Part 3: Google Firebase & Apache Found Vulnerable to Environment Abuse](#)

What Are Artifacts?

Artifacts are the products of the work done in the CI/CD pipelines and are usually used to:

-

Persist workflow data after a job has been completed for later use, such as log files, core dumps, test results, pictures, etc.

-

A communication channel between jobs in the same workflow (usually this is used to optimize the workflow run time), for example, assume you have a workflow named 'build_and_test.yml' which contains two jobs which are surprisingly called 'build' and 'test,' instead of building the project twice, which is time-consuming, the workflow author could compile and upload the build artifact from the 'build' job and then download it in the 'test' job to continue the processing.

The rust-lang Vulnerability

"rust-lang" is the official Rust project in GitHub, and the vulnerability we found is in the "rust-lang/rustc_codegen_gcc" repository, and could allow any user to **execute code in a privileged pipeline**. This gave any user the ability to write code that extracts repository secrets like cloud credentials, modifies project settings and even tampers with the source code using GitHub API, since the privileged pipeline has access to all that.

The vulnerability was found in a workflow called "ci.yml" which is responsible for building and testing the repository's code. Below is the vulnerable part of the workflow (you can find the full workflow [here](#)).

[\[image: Vulnerable part of a workflow - Novel Pipe Vulnerability\]](#)

In step "Download artifact", the workflow downloads the "libgccjit.so" library from the "antoyo/gcc" repository and in step "Set env" it sets the runner to use it by modifying the linker environment variables.

If we could trick the workflow into downloading our own "libgccjit.so" library,*
we could make the runner use our library and execute arbitrary code in the workflow. Let's

see how that is possible.*

The “[action-download-artifact](#)” action downloads the **latest** artifact named `$$` from another repository, named “[antoyo/gcc](#)”, as part of “[main.yml](#)” workflow. Surprisingly it doesn't distinguish between artifacts that were created by **forked** versions of “antoyo/gcc” repository and the original one.

To initiate the attack, we simply need to:

-

Fork “antoyo/gcc”

-

Modify “main.yml” to upload our malicious version of “libgccjit.so”

-

Create a pull request from our fork to “antoyo/gcc”

Consequently, we trick the rust workflow into pulling our poisoned artifact and executing it in the subsequent steps of the workflow.

(Deep dive into the root cause of the vulnerability is presented in the next section of this blog)

[image: image]

[[image: GitHub Artifact Poisoning Map](#)]*GitHub Artifact Poisoning Map*

Artifact Poisoning Demo

Watch a short demonstration of an artifact being poisoned with a malicious code injection to better understand how a bad actor can compromise your privileged pipelines using the techniques covered in this blog.

[[image: Legit Security - Noam Rust Hack Video](#)]

(<https://www.legitsecurity.com/hubfs/Video/Noam%20Rust%20Hack%20Video.mp4>)

[Watch A Video Demonstration of A Malicious Injection](#)

Impact

Upon exploitation, the attacker could modify the repository branches, pull requests, issues, releases, and all of the entities that are available for the workflow token permissions. Since rust didn't limit the workflow token for specific scopes, it's all of the below:

[[image: scopes for rust token that can be compromised via artifact poisoning attack](#)]

Depending on Rust's specific configuration, a skilled attacker could probably use these permissions to extend the attack outside the vulnerable repository to additional Rusts assets and move laterally inside the organization.

Disclosure Timeline

-

5.4.22 Reported to GitHub

-

29.4.22 GitHub acknowledged the issue

-

28.6.22 GitHub Updated their API + payout

-

15.09.2022: Reported to Rust

-

15.09.2022: Rust security team acknowledged the report and temporarily disabled GitHub Actions for that repository

-

16.09.2022: Rust security team analyzed the vulnerability and suggested a fix

-

16.09.2022: We provided more details and confirmed that the fix was correct

-

26.09.2022: Rust security team fixed the issue ([see here](#)).

Diving In

GitHub Artifacts storage:

The artifacts storage is divided into buckets. Each workflow run (an instance of workflow file invocation) is assigned a unique bucket to which only jobs within that workflow can upload artifacts to (the workflow is identified by its run_id).

Uploading an artifact:

[[image: Uploading an artifact in Github - Novel Pipeline Vulnerability](#)]

Downloading an artifact:

[[image: Downloading an artifact in Github - Novel Pipeline Vulnerability](#)] These actions use the bucket's API to upload & download artifacts from the currently assigned bucket.

The limitation and the workaround:

The above actions don't allow downloading artifacts created in different workflows (cross-workflow). Yet, some workflows need to use artifacts created in different workflows to work. For example, imagine a release workflow that combines artifacts from several workflows to create the final release:

-

Download frontend artifacts

-
Download backend artifacts

-
Download generated docs

-
Build final release

This is not possible with the native GitHub-provided actions.

To overcome this limitation, workflow authors started to use [GitHub-provided API](#) that allows downloading artifacts from other workflows based on some filtering capabilities (i.e., artifact created by a specific workflow file, specific user, specific branch, etc.).

[\[image: Github-Provided API - Novel Pipeline Vulnerability\]](#)

The problem

The “download artifacts” API (and various custom actions encapsulating it) doesn’t differentiate between artifacts that were uploaded by forked repositories and base repositories, which could lead privileged workflows to download artifacts that were created by forked repositories and that are potentially poisoned.

To put it simply: in a vulnerable workflow, any GitHub user can create a fork that builds an artifact. Then inject this artifact into the original repository build process and modify its output. This is another form of a software supply chain attack, where the build output is modified by an attacker.

Additionally, the lack of native GitHub implementation for cross-workflow artifacts communication led many projects and the GitHub Actions community to develop insecure solutions for cross-workflow communication and made this threat highly prevalent.

Example:

One of the most common custom actions that aim to overcome this limitation is [dawidd6/action-download-artifact](#), which has above 12K dependent repositories.

Let's show how it can make your workflow vulnerable: The following workflow is a manually triggered workflow that pulls the last built image from the ‘build.yml’ workflow and publishes it (copied from a real repository):

[\[image: Manually triggered workflow to pull and publish the last built image of a workflow\]](#)

Now, since the workflow author doesn’t specify explicitly from which run id the artifact should be downloaded, an adversary could take the following actions to inject a malicious artifact:

-
Fork the repository.

-

Create the following 'build.yml' workflow file:[image: Potential action from a malicious artifact]

- Open a pull request to the base repository.

Once the manual workflow runs, it will download and publish the malicious artifact instead of the intended one. This is because 'action-download-artifact' is not limited to the current bucket limitation, and we uploaded an artifact that matches the provided search criteria.

Additional vulnerable download-artifact actions:

It appears that the vast majority of 'download-artifact' custom actions are vulnerable. Here are a few examples:

- <https://github.com/aochmann/actions-download-artifact>
- <https://github.com/bettermarks/action-artifact-download/blob/master/main.py>
- <https://github.com/levonet/action-download-last-artifact>
- <https://github.com/dawidd6/action-download-artifact>

Suggested mitigations:

1- Specify which run id or commit hash to download the artifact from.

[image: Suggested mitigation to prevent actions from malicious artifacts]

[image: Suggested mitigation 1: Specify which run id to download the artifact from.]

2- Filter out artifacts that were created by pull-request.

[image: Suggested mitigation 2 to prevent actions from malicious artifacts]3- Never trust the content of cross-workflow artifacts. Always sanitize the payload before using it. For example, if you expect a pull-request number make sure the value is a number.

4- Make sure you control which workflows are allowed to run by enabling "Require approval for all outside collaborators" in Repository → Settings → Actions → General.

Report to GitHub:

This security issue was reported to GitHub through their bug bounty program. They acknowledged it and updated the GetArtifact and ListArtifacts APIs to provide more information that would help developers differentiate between trusted artifacts and untrusted ones. Yet, they decided that the fundamental issue of cross-workflow artifacts is not a significant risk and should be handled by the workflow authors. So once again, it's up to the repositories maintainers to make sure they are safe. We highly encourage workflow authors that use GitHub artifacts across different workflows to use the suggested mitigations.

How Legit Security Can Help
